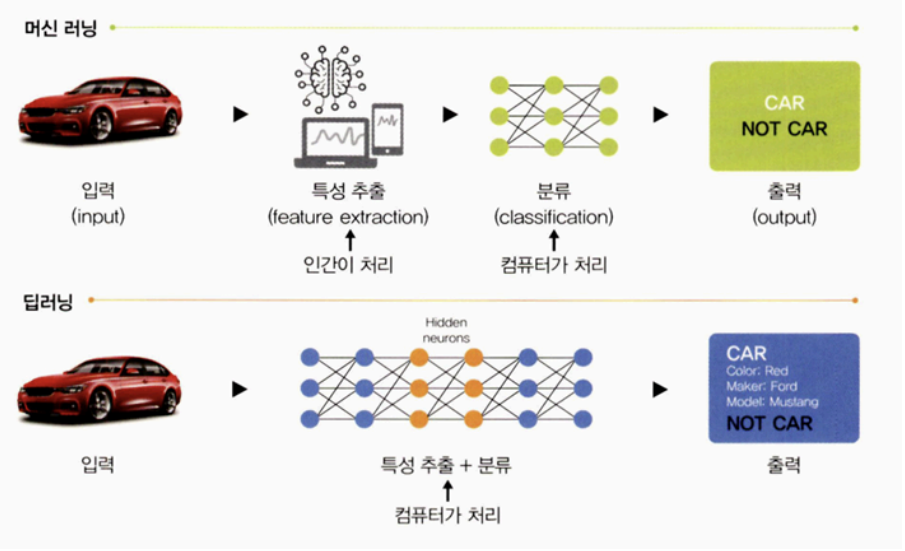
1. 머신러닝과 딥러닝

1.1 인공지능, 머신러닝과 딥러닝

인공지능 : 인간의 지능을 모방하여 사람이 하는 일을 기계가 할 수 있도록 하는 기술

- 구현 방법 : 머신러닝, 딥러닝
  - 머신러닝 : 주어진 데이터를 인간이 먼저 처리, 즉 각 데이터 특성을 기계에 인식시키고 학습시켜 문제 해결 (데이터의 특징을 스스로 추출 X)
  - 딥러닝 : 인간이 하던 작업 생략, 대량의 데이터를 신경망에 적용하면 컴퓨터가 스스로 분석 후 답을 찾음

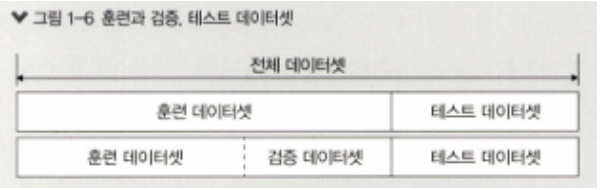


1.2 머신러닝이란

1.2.1 머신러닝 학습과정

- 학습 단계 : 훈련 데이터를 머신러닝 알고리즘에 적용해 학습
- 예측 단계 : 학습단계에서 생성된 모형에 새로운 데이터를 적용하여 결과 예측
- 주요 구성 요소
  - 데이터 : 학습 모델에 필요, 훈련 데이터셋 / 테스트 데이터셋으로 분리
  - 모델 : 머신러닝의 학습 단계에서 얻은 최종 결과물 (가설)
    - 모델 학습 절차 : 모델/가설 선택 → 모델 학습 및 평가 → 평가를 바탕으로 모델 업데이트

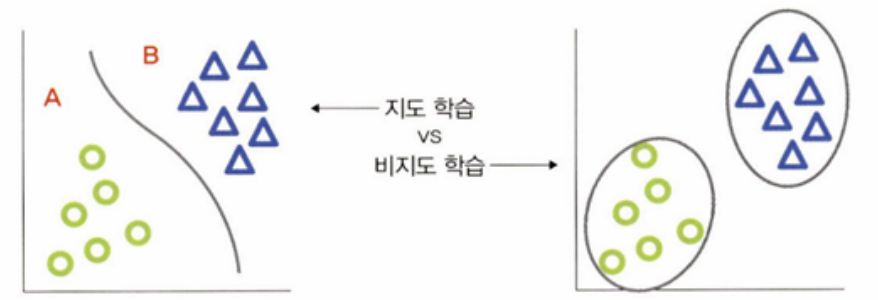
- 훈련과 검증, 테스트 데이터셋



→ 훈련 데이터셋을 종종 훈련 & 검증(validation) 용도로 분리하여 사용 : 모델 성능을 평가하기 위해

1.2.1 머신러닝 학습 알고리즘

- 지도 학습 : 정답이 무엇인지 컴퓨터에 알려주고 학습
- 비지도학습 : 정답을 알려주지 않고 특징이 비슷한 데이터를 클러스터링하여 예측하는 학습 방법

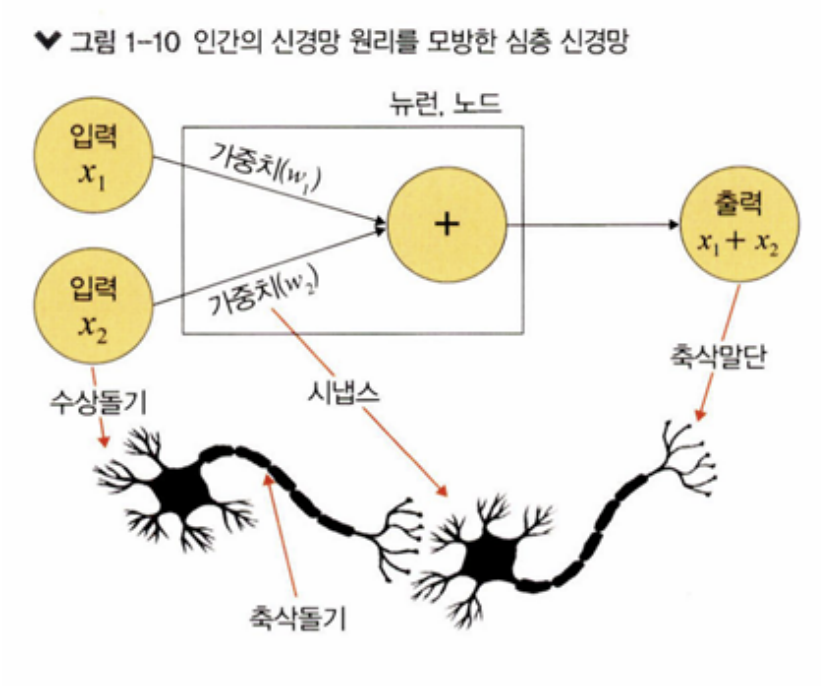


- 강화학습 : 자신의 행동에 대한 보상을 받으며 학습을 진행 → 보상이 커지는 행동은 자주 하도록 하고, 줄어드는 행동을 덜 하도록

1.3 딥러닝이란

딥러닝 : 인간의 신경망 원리를 모방 → 인간의 뇌를 기초로 해서 설계됨

- 뉴런, 시냅스



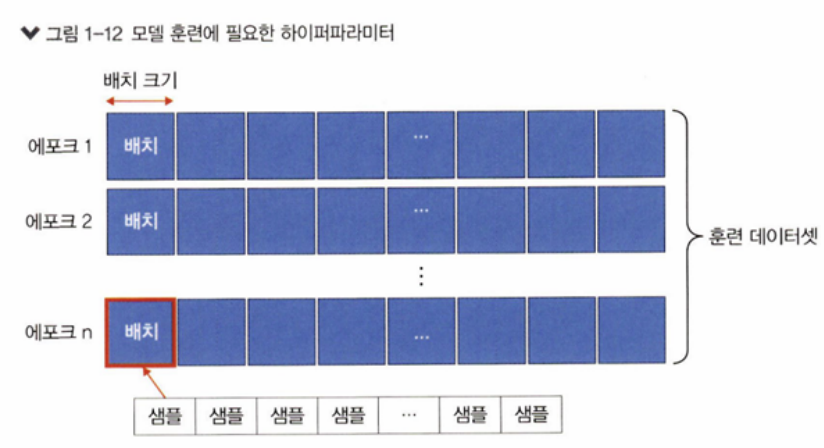
- 수상돌기 : 주변이나 다른 뉴런에서 자극을 받아들임
- 시냅스 : 한 뉴런의 축삭돌기와 다음 뉴런의 수상돌기가 만나는 부분
- 축삭돌기 : 다른 뉴런(수상돌기)에 신호를 전달
- 축삭말단 : 전달된 전기신호를 받아 신경전달 물질을 시냅스 틈새로 방출

1.3.1 딥러닝 학습 과정

1. 데이터 준비
   a. 파이토치, 케라스에서 제공하는 데이터셋 사용
   b. 캐글 등 공개된 데이터셋 사용
2. 모델(모형) 정의
   a. 신경망 생성 : 일반적으로 은닉층 개수가 많을수록 성능이 좋아지지만, 과적합 발생 확률도 높아짐
3. 모델 컴파일 : 활성화 함수, 손실함수, 옵티마이저 선택
   a. 데이터셋이 연속형 → MSE(평균제곱오차) 사용
   b. 데이터셋이 이진분류 → Cross entropy 사용
4. 모델 훈련 : 한번에 처리할 데이터양 지정

a. 메모리 → 전체 훈련 데이터셋에서 일정한 묶음으로 나누어 처리할 수 있는 배치와 훈련 횟수 선택

   i. 하이퍼 파라미터, 파라미터 최적값 찾기

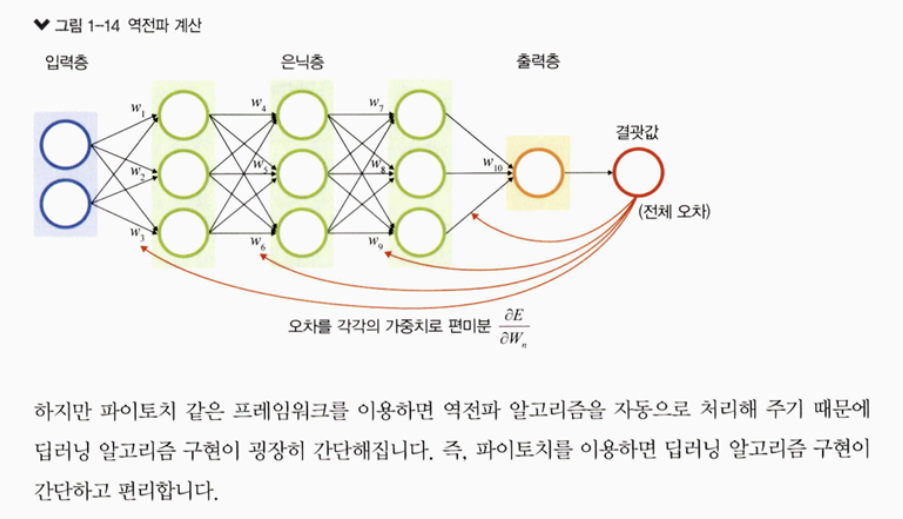


ex) 훈련 데이터셋 1000개에 대한 배치 크기가 20 → 샘플 단위 20개마다 모델 가중치를 한번씩 업데이트 → 총 50번(1000/20)의 가중치 업데이트

5. 모델 예측 : 검증 데이터셋을 생성한 모델에 적용해 실제로 예측 진행

중요한 요소들

- 심층 신경망 : 데이터셋의 어떤 특성들이 중요한지 가르쳐 줄 수 있음
- 역전파 : 가중치 값 업데이트



1.3.2 딥러닝 학습 알고리즘

지도 학습 / 비지도 학습/ 전이 학습

1. 지도 학습

- 합성곱 신경망(CNN) : 이미지 분류, 이미지 인식, 이미지 분할
- 순환 신경망(RNN) : 시계열 데이터 분류에 사용 → 역전파 과정에서 기울기 소멸 문제를 해결하기 위해 LSTM

2. 비지도학습

- 워드 임베딩 : 단어를 벡터로 표현
- 군집 : 아무런 정보가 없는 상태에서 데이터를 분류
    - 한 클러스터 내의 데이터는 비슷하게, 다른 클러스터의 데이터와는 다르게 구분하는 것이 목표

3. 전이 학습

   : 사전에 학습이 완료된 모델을 가지고 원하는 학습에 미세조정 기법을 이용해 학습시키는 방법

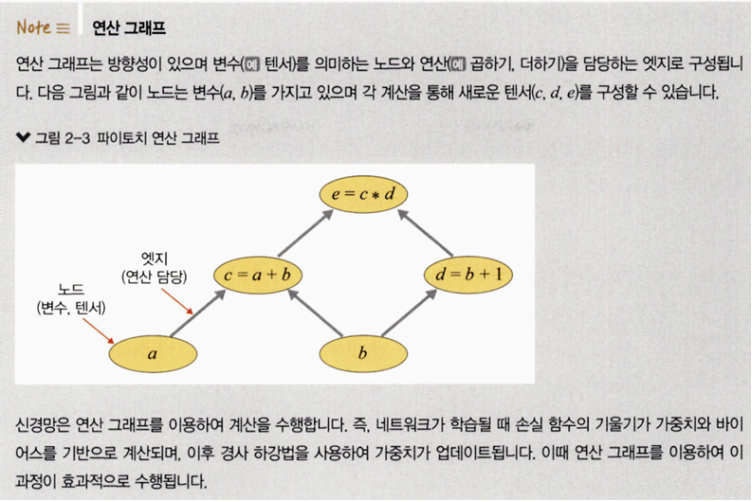
   - 사전 학습 모델 : 풀고자 하는 문제와 비슷하면서 많은 데이터로 이미 학습이 되어있는 모델
     - ex. VGG, MobileNet, 인셉션

2. 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

2.1.1 파이토치 특징 및 장점

- GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크
  - GPU : 연산 속도를 빠르게 함
  - TENSOR : 파이토치의 데이터 형태로, 단일 데이터 형식으로 된 자료들의 다차원 행렬
  - 동적 신경망 : 훈련을 반복할 때마다 네트워크 변경이 가능한 신경망
    - Define by Run : 연산 그래프를 정의하는 것과 동시에 값도 초기화됨



Note ≡ **연산 그래프**

연산 그래프는 방향성이 있으며 변수(텐서)를 의미하는 노드와 연산(곱하기, 더하기)을 담당하는 엣지로 구성됩니다. 다음 그림과 같이 노드는 변수(a, b)를 가지고 있으며 각 계산을 통해 새로운 텐서(c, d, e)를 구성할 수 있습니다.

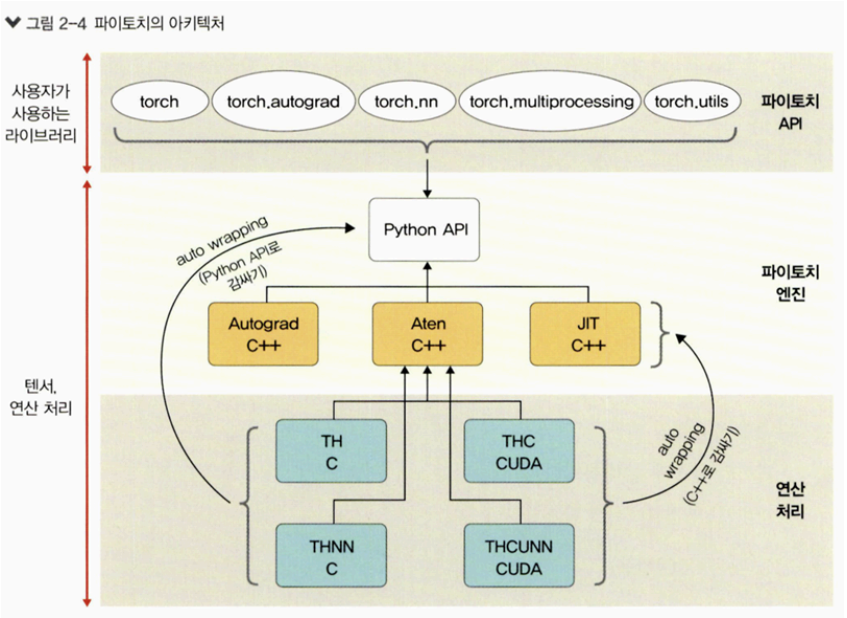
❮ 그림 2-3 파이토치 연산 그래프

신경망은 연산 그래프를 이용하여 계산을 수행합니다. 즉, 네트워크가 학습될 때 손실 함수의 기울기가 가중치와 바이어스를 기반으로 계산되며, 이후 경사 하강법을 사용하여 가중치가 업데이트됩니다. 이때 연산 그래프를 이용하여 이 과정이 효과적으로 수행됩니다.

- 파이토치의 장점
  - 효율적인 계산, 낮은 CPU 활용, 직관적인 인터페이스와 낮은 진입장벽

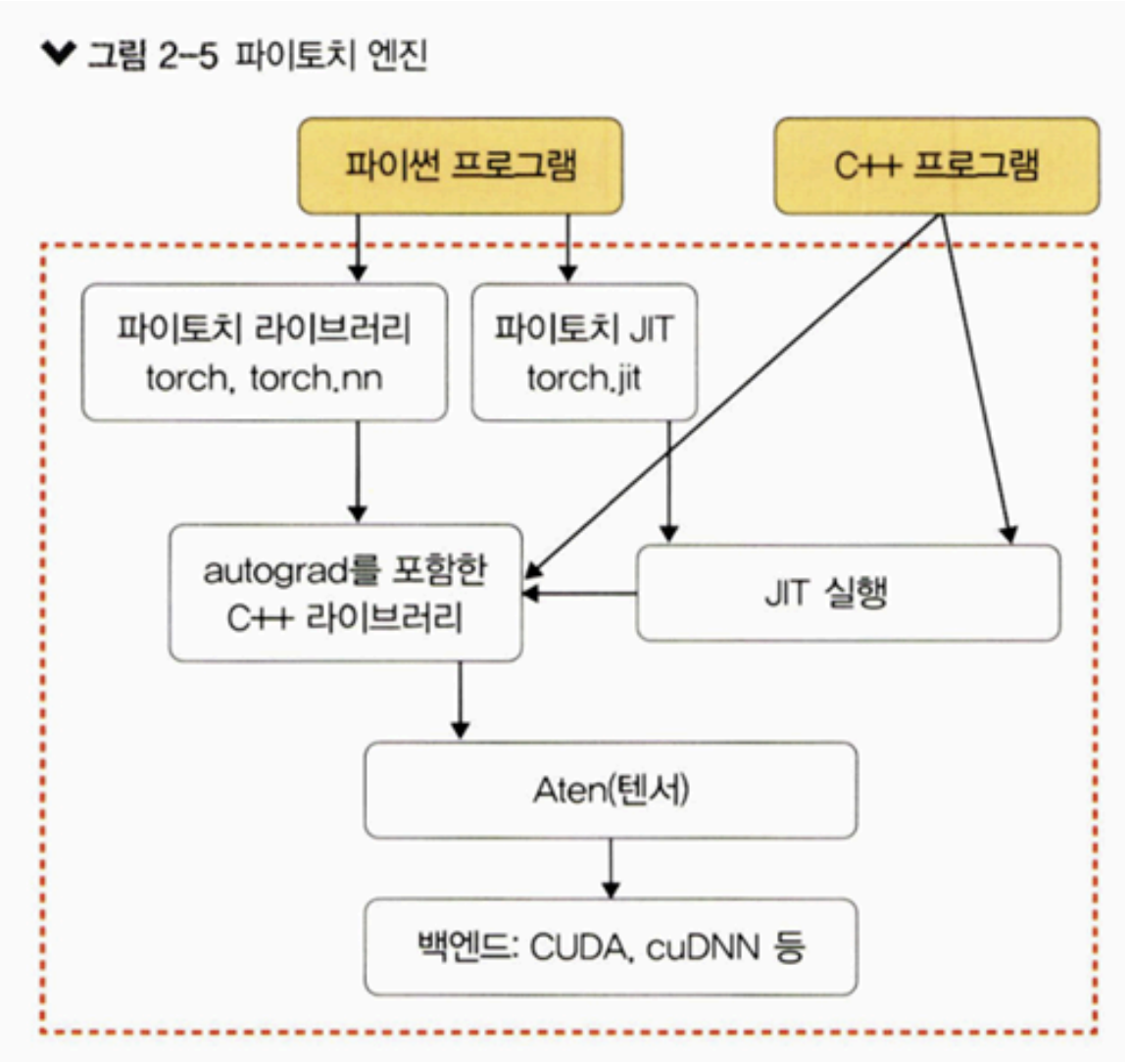
2.1.1 파이토치의 아키텍처

: 파이토치 API(상위), 파이토치 엔진(중간), 텐서 연산처리(하위)



1. 파이토치 API : 사용자가 이해하기 쉬운 API를 제공 → 텐서에 대한 처리와 신경망을 구축하고 훈련할 수 있도록 함.
    - 실제계산 X, 작업을 전달하는 역할
    - torch : CPU를 지원하는 텐서 패키지
    - torch.autograd : 자동 미분(auto-differentiation) 패키지
    - torch.nn : 신경망 구축 및 훈련 패키지
    - torch.multiprocessing : 파이토치에서 사용하는 프로세스 전반에 걸쳐 메모리 공유 가능
    - torch.utils : DataLoader 및 기타 유틸리티 제공
2. 파이토치 엔진

    : Autograd C++, Aten C++, JIT C++, Python API로 구성
    - Autograd C++ : 가중치, 바이어스를 업데이트하는 과정에서 필요한 미분을 자동으로 계산
    - Aten C++: 텐서 라이브러리 제공
    - JIT C++ : 계산 최적화를 위한 컴파일러
    - Python API : 파이토치 엔진 라이브러리를 C++로 래핑한 다음 제공되는 형태 → 사용자가 손쉽게 모델을 구축하고 텐서 사용 가능



3. 연산 처리 (p.38)

- C, CUDA 패키지는 상위 API에서 할당된 거의 모든 계산을 수행
- 오프셋 , 스트라이드 → 데이터 자체가 아니라 행렬/텐서를 구분하기 위해 사용

2.2 파이토치 기초문법

2.2.1 텐서 다루기

- 텐서 생성 및 변환
  - 텐서 ndarray 변환
- 텐서 인덱스 조작
  - 파이토치로 1차원 벡터 생성 → 인덱스로 접근 → 슬라이스로 접근
- 텐서 연산 및 차원 조작

    - 벡터 생성 → 사칙 연산 가능
    - 텐서 차원 조작
        - view, stack, cat, t, transpose 사용

2.2.2 데이터 준비

→ 판다스, 파이토치 제공 데이터 이용

- 데이터가 이미지 : 분산된 파일에서 데이터를 읽음 → 전처리 → 배치 단위로 분할하여 처리
- 데이터가 텍스트 : 임베딩 과정 → 서로 다른 길이의 시퀀스를 배치 단위로 분할하여 처리
- 파일을 불러오거나, 커스텀 데이터셋을 만들어서 사용
    - 커스텀 데이터셋 : 데이터를 한번에 다 부르지 않고 조금씩 나누어 불러서 사용하는 방식

```
class CustomDataset(torch.utils.data.Dataset):
    def __init__(self) :
    def __len__(self):
    def __getiem__(self, index):
```

- 파이토치에서 제공하는 데이터셋 사용
    - 토치비전(torchvision) : MNIST, ImageNet 등 유명한 데이터셋 제공

2.2.3 모델 정의

- 모델을 정의하기 위해서는 모듈을 상속한 클래스 사용
- 계층(layer) : 모듈 또는 모듈을 구성하는 한 개의 계층으로 합성곱층, 선형 계층 등 존재
- 모듈 : 한 개 이상의 계층이 모여서 구성된 것으로, 모듈이 모여 새로운 모듈을 만들 수 있음
- 모델 : 최종적으로 원하는 네트워크로, 한 개의 모듈이 모델이 될 수도 있음

```
model = nn.Linear(in_features = 1, out_features = 1, bias =
True)
```

- nn.Module()을 상속하여 정의하는 방법
    - __init__(), forward() 함수 포함

- __init__() : 모델에서 사용될 모듈, 활성화 함수 등 정의
- forward(): 모델에서 실행되어야 하는 연산 정의

```
class MLP(Module):
    def __init__(self, inputs):
        super(MLP, self).__init__()
        self.layer = Linear(inputs,1 )
        self.activation = Sigmoid()

    def forward(self, X):
        X = self.layer(X)
        X = self.activation(X)
        return X
```

- Sequential 신경망 정의하는 법

```
import torch.nn as nn
class MLP(nn.Module):
    def __init__(self, inputs):
        super(MLP, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels = 3, out_channels = 64, k
ernel_size = 5),
            nn.ReLU(inplace = True),
            nn.MaxPool2d(2)
        )

        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels = 64, out_channels = 30,
kernel_size = 5),
            nn.ReLU(inplace = True),
            nn.MaxPool2d(2)
        )

        self.layer3 = nn.Sequential(
            nn.Linear(in_features = 30*5*5, out_features =
10, bias = True),
            nn.ReLU(inplace = True)
```

```
        )

    def forward(self, x):
        X = self.layer1(x)
        X = self.layer2(x)
        x = x.view(x.shape[0], -1)
        x = self.layer3(x)
        return X

model= MLP()

print("Printing children\n--------------------------")
print(list(model.children()))
print("\n\nPrinting Modules\n--------------------------")
print(list(model.modules()))
```

→ nn.Sequential은 모델의 계층이 복잡할수록 효과가 뛰어남

- 함수로 신경망을 정의하는 방법
  - Sequential을 이용하는 것과 동일하지만, 함수로 선언할 경우 변수에 저장해둔 계층들을 재사용 가능

2.2.4 모델 파라미터 정의

- 사전 정의 파라미터
  - 손실 함수 : 학습하는 동안 출력과 실제값 차이의 오차를 측정
    - BCELoss : 이진 분류
    - CrossEntropyLoss : 다중 클래스 분류
    - MSELoss : 회귀 모델에서 사용
    - 전역 최소점과 최적점
      - 전역 최소점 : 오차가 가장 작을 때의 값
      - 지역 최소점 : 최소점을 찾아가는 과정에서 만나는 곳, 옵티마이저가 지역 최소점에서 학습을 멈추면 최솟값을 갖는 오차를 찾을 수 없는 문제 발
  - 옵티마이저 : 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법을 결정
    - step() 메서드를 통해 전달받은 파라미터 업데이트
    - 모델의 파라미터별로 다른 기준을 적용

- torch.optim.Optimizer(params, defaults)는 모든 옵티마이저의 기본이 되는 클래스
- zero_grad() 메서드 : 옵티마이저에 사용된 파라미터들의 기울기를 0으로
- torch.optim.lr_schedulrer는 에포크에 따라 학습률 조절
- 학습률 스케줄러 : 미리 지정한 횟수의 에포크를 지날 때마다 학습률을 감소시켜줌
- 지표 (metrics) : 훈련과 테스트 단계를 모니터링

```
from torch.optim import optimizer
criterion = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr = 0.0
1, momentum = 0.9)
scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer
= optimizer,
                                                lr_lambda
= lambda epoch : 0.95 ** epoch)
for epoch in range(1, 100+1):
    for x,y in dataloader:
        optimizer.zero_grad()

loss_fn(model(x), y).backward()
optimizer.step()
scheduler.step()
```

2.2.5 모델 훈련

- 데이터로 모델 학습
  - y=wx_b에서 w,b의 적절한 값을 찾음
- optimizer.zero_grad() 메서드를 이용해 기울기를 초기화
- loss.backward() → 새로운 기울기값이 이전 기울기값에 누적해 계산됨
  - 기울기 값에 대해 누적 계산이 필요하지 않다면 입력값을 모델에 적용하기 전에 optimizer.zero_grad() 메서드를 호출해 미분값이 누적되지 않게 초기화해야 함

```
for epoch in range(100):
    yhat = model(X_train)
    loss = criterion(yhat, y_train)
```

```
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

2.2.6 모델 평가

- 주어진 테스트 데이터셋을 사용해 모델 평가

```
import torch
import torchmetrics
metric = torchmetrics.Accuracy()

n_batches = 10
for i in range(n_batches):
    preds = torch.randn(10,5).softmax(dim=-1)
    target = torch.randint(5, (10,))

    acc = metric(preds, target)
    print(f"Accuracy on batch {i} :{acc}")
acc = metric.compute()
print(f"Accuracy on all data : {acc}")
```

- 혼동행렬 이용 가능

2.2.7 훈련 과정 모니터링

- 텐서보드 이용 : 학습에 사용되는 각종 파라미터 값이 어떻게 변화하는지 살펴볼 수 있음
    - 텐서보드 설정 → 텐서보드에 기록 → 텐서보드를 사용해 모델 구조 살피기

```
import torch
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter("../chap02/tensorboard")

for epoch in range(num_epochs):
    model.train()
    batch_loss = 0.0

    for i, (x, y) in enumerate(dataloader):
        x,y = x.to(device).float(),y.to(device).float()
        outputs = model(x)
```

```
        loss = criterion(outputs, y)
        writer.add_scalar("Loss", loss, epoch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

writer.close()
```

```
tensorboard --logdir=../chap02/tensorboard --port=6006
```

- model.train() : 훈련 데이터셋에 사용, 모델 훈련이 진행될 것임을 알림
- model.eval() : 모델을 평가할 때는 모든 노드를 사용하겠다는 의미로 검증과 테스트 데이터셋에 사용

2.3 실습환경 설정

2.3.1 아나콘다 설치

→ 아나콘다 다운로드 : 웹페이지 다운로드

2.3.2 가상 환경 생성 및 파이토치 설치

- 가상환경 생성
  - Anaconda Prompt 이용
- 파이토치 설치
  - 아나콘다 프롬프트 이용
    - conda install pytorch ==1.9.0 torchvision = 0.10.0 torchaudio = 0.9.0 -c pytorch
- 구글 코랩 사용

2.4 파이토치 코드 맛보기

범주형 데이터 텐서 변환

- 범주형 데이터 → dataset → 넘파이 배열 → 텐서
- get_dummies : 가변수로 만들어주는 함수 (문자를 숫자 0,1로 바꾸어줌
- True positive : 모델이 1이라고 예측했는데 실제값도 1

- True negative : 모델이 0이라고 예측했는데 실제값도 0
- True negative : 모델이 1이라고 예측했는데 실제값은 0
- False negative : 모델이 0이라고 예측했는데 실제값은 1
- 정확도 : 전체 예측 건수에서 정답을 맞힌 건수의 비율

$$\frac{True\ Positive + True\ Negative}{True\ Positive + True\ Negative + False\ Positive + False\ Negative}$$

- 재현율 : 실제로 정답이 1이라고 할 때 모델도 1이라고 예측한 비율

$$\frac{True\ Positive}{True\ Positive + False\ Negative}$$

- 정밀도 : 모델이 1이라고 예측한 것 중에서 실제로 정답이 1인 비율

$$\frac{True\ Positive}{True\ Positive + False\ Positive}$$

- F1 스코어 : 정밀도와 재현율의 조화평균

$$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$